

打造四軸飛行器

黃大峯

今天會提到的

- 解釋基本四軸飛行器原理
- 濾波器以及控制器設計
- 在即時作業系統(RTOS)上運行飛行控制

飛行是人類自古以來的夢想



1903年12月17日萊特兄弟試飛成功

世界各地的開源飛控板

Paparazzi Project

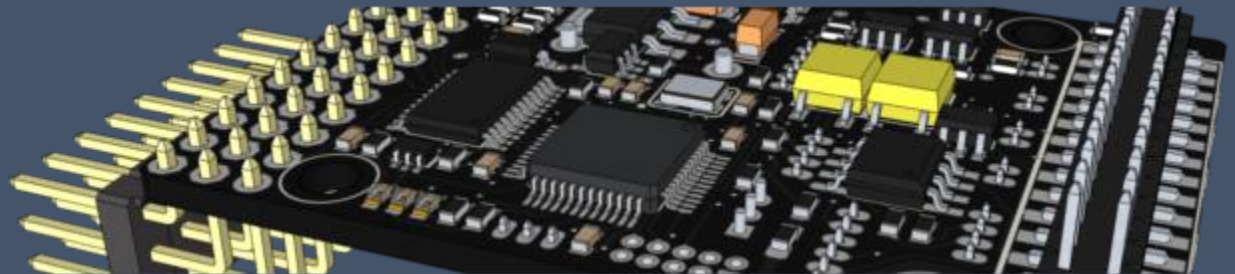
- École nationale de l'aviation civile (ENAC), French civil aeronautics academy
- Began in 2003
- Support various hardware
- RT Paparazzi: ChibiOS/RT (*rt_paparazzi* branch)

Short video about open uav: <http://youtu.be/lePG2fzKvF0>

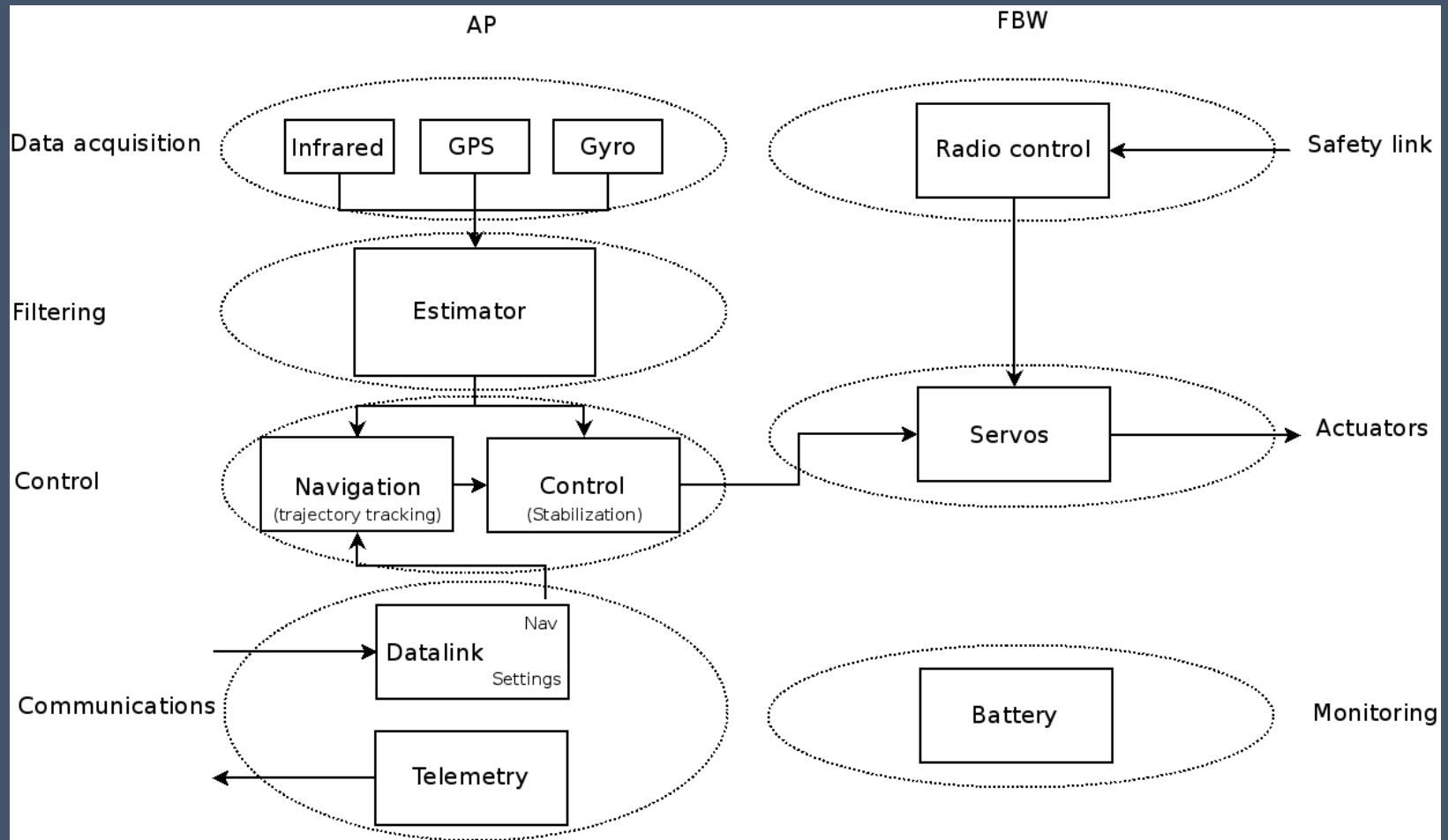


PX4 Autopilot

- Started in 2009
- Being further developed and used at Computer Vision and Geometry Lab of ETH Zurich (Swiss Federal Institute of Technology) and supported by the Autonomous Systems Lab and the Automatic Control Laboratory
- RTOS: NuttX



飛控板做了些什麼？



涵蓋的技術及理論

- 基礎數學及物理
- 力學
- 線性代數
- 微積分
- 電子電路學
- 通訊系統
- 數位訊號處理
- 控制系統
- 作業系統

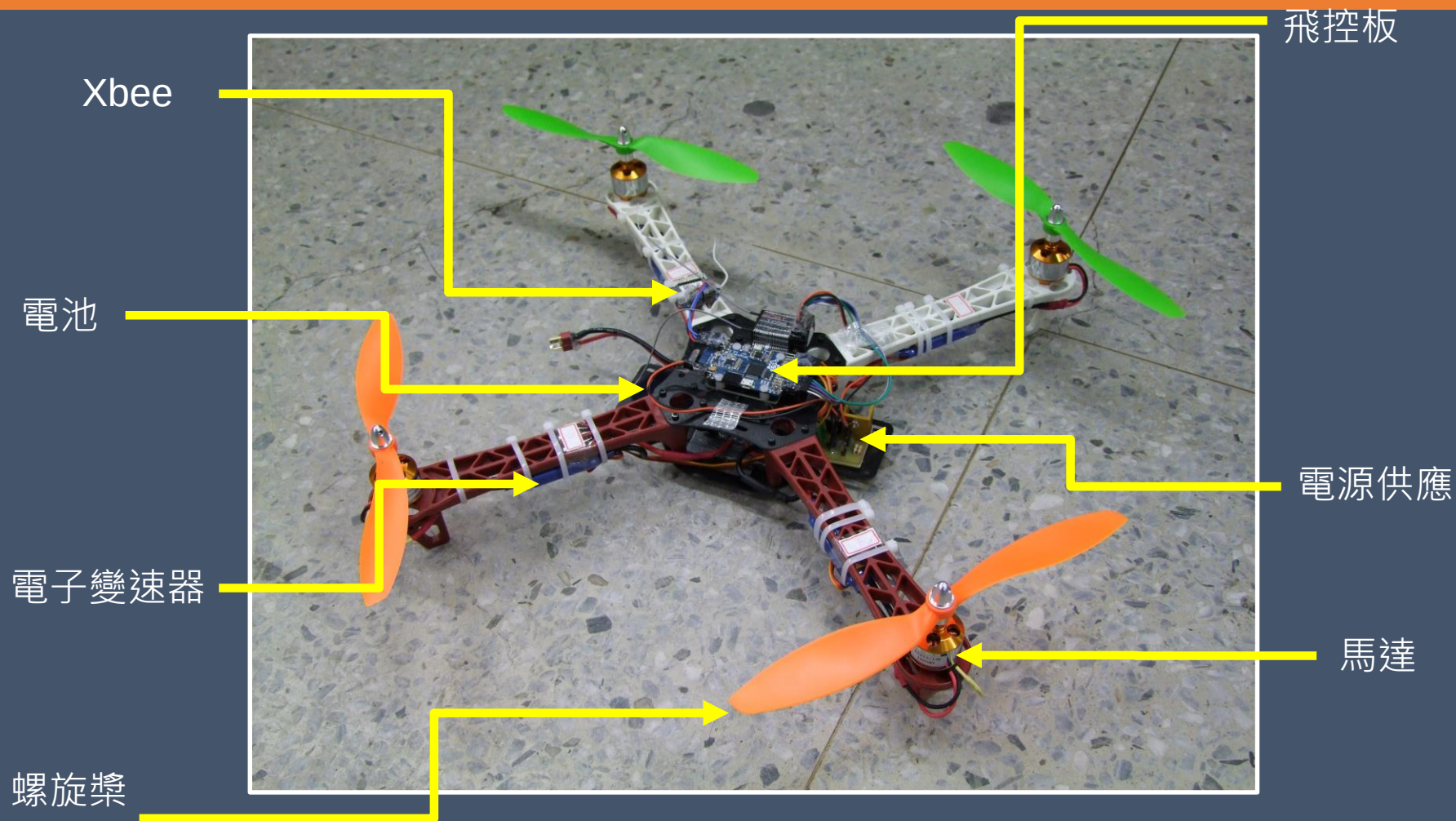
各式各樣的飛行器



我們所打造的飛行器

如何(幾乎)從零打造一台四軸飛行器？

硬體外觀

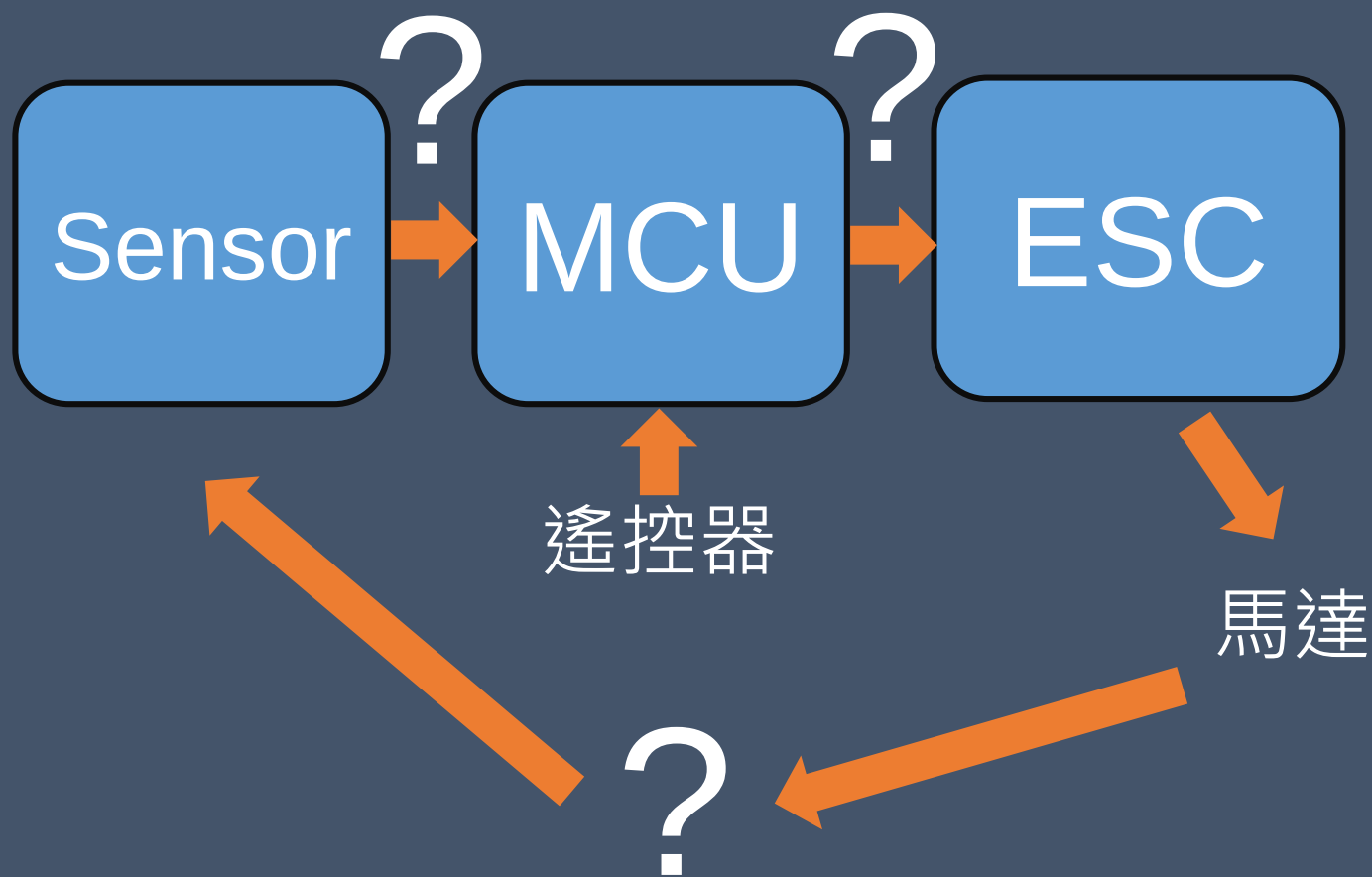


組裝

- 轉向
 - 槳葉有分正反(順逆時針轉向)，馬達也有自己的轉向，必須兩個都同向
- 配重
 - 盡量維持重心在正中央
- 固定
 - 確認所有東西都有螺絲鎖上，可以的話用熱溶膠和螺絲防鬆劑
- 配電
 - 用**電源供應器**確認電路是否有問題
- 起飛前確認沒有東西會甩出去
 - **膠帶**是你的好朋友

飛的條件

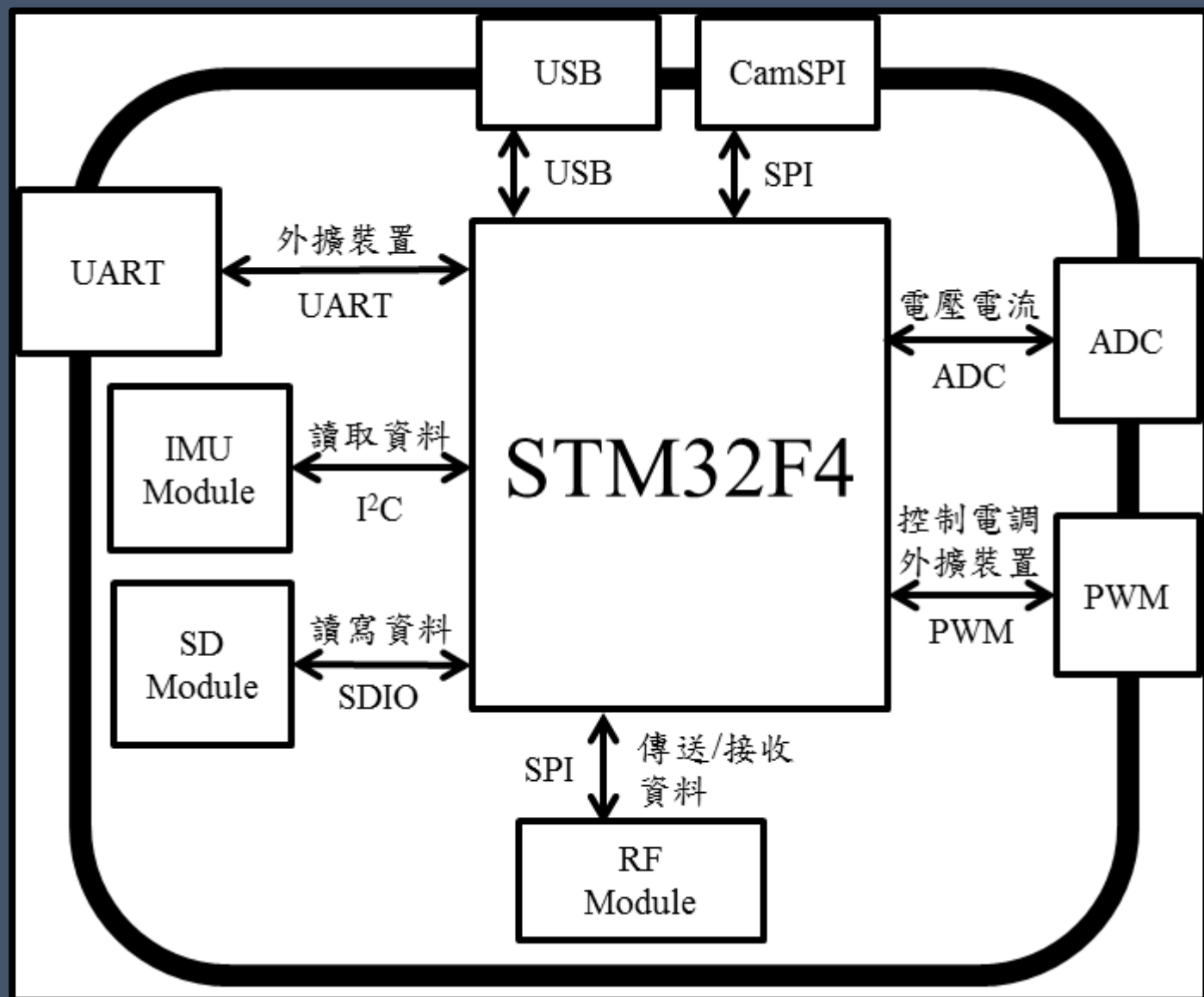
- 讓馬達可以根據我們感測的訊息來轉動



飛控板

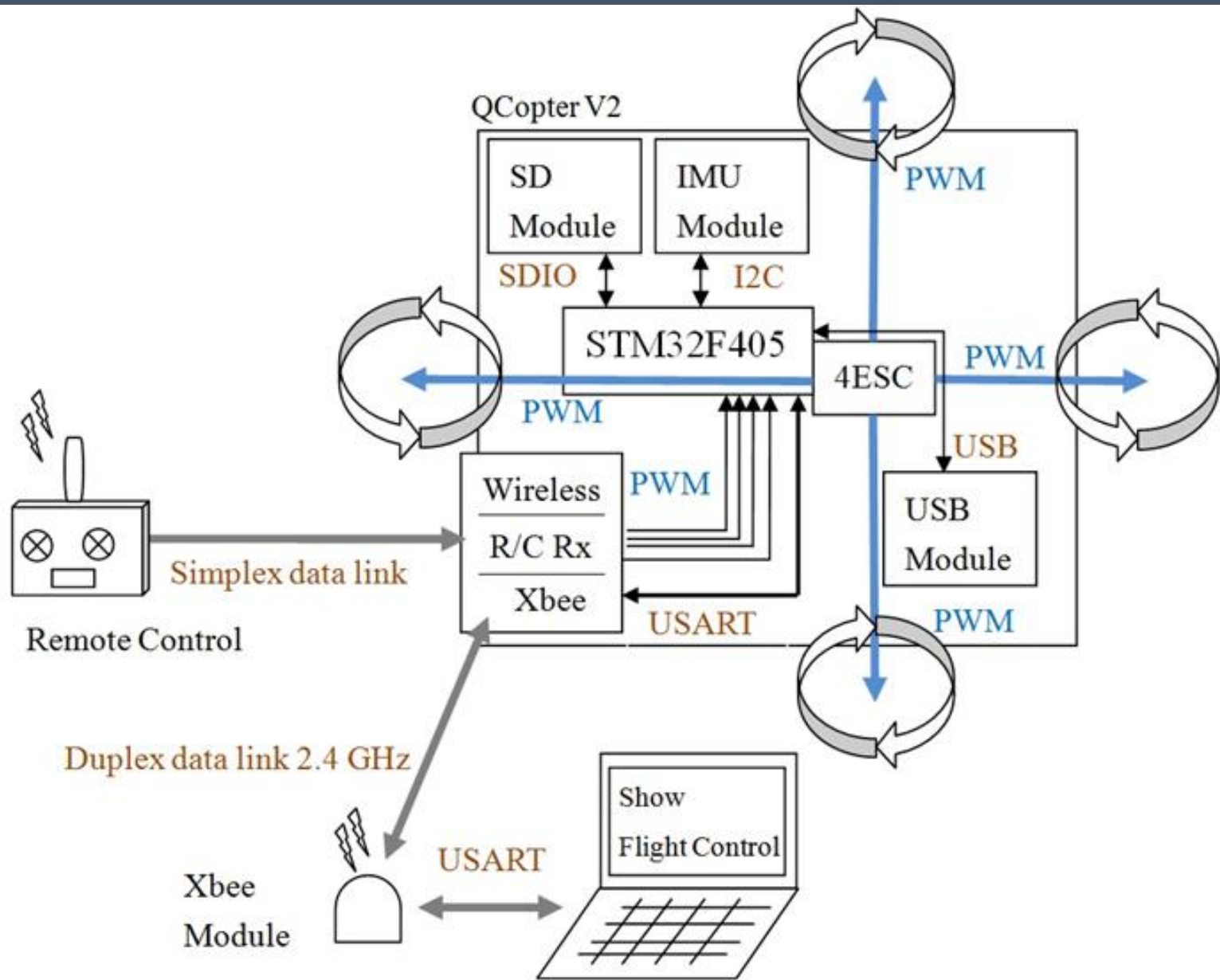
- 控制器：STM32F405R 64Pin 168MHz DSP FPU
- 感測器：IMU_10-DOF (MPU-9150 + MS5611)
- 無線傳輸：nRF24L01P + PA + LNA
- 儲存紀錄：Micro SD，使用 SDIO 操作
- 外接介面：2*ADC、1*USB (Micro)、1*SPI (FFC16)、1*UART、12*PWM
- PCB 尺寸：52 * 35mm (Screws M3: 30 * 30mm)

*由中原大學的王文宏同學所親自設計及焊接



飛控板

- 我們如何利用這些IO？
 - 遙控器訊號輸入(PWM Input)
 - 五個Timer通道
 - 馬達訊號輸出(PWM Output)
 - 四個Timer通道
 - Xbee
 - 以USART無線傳輸
 - nRF24L01
 - MCU利用SPI介面傳送至模組，再利用無線傳輸至接收端



基本原理

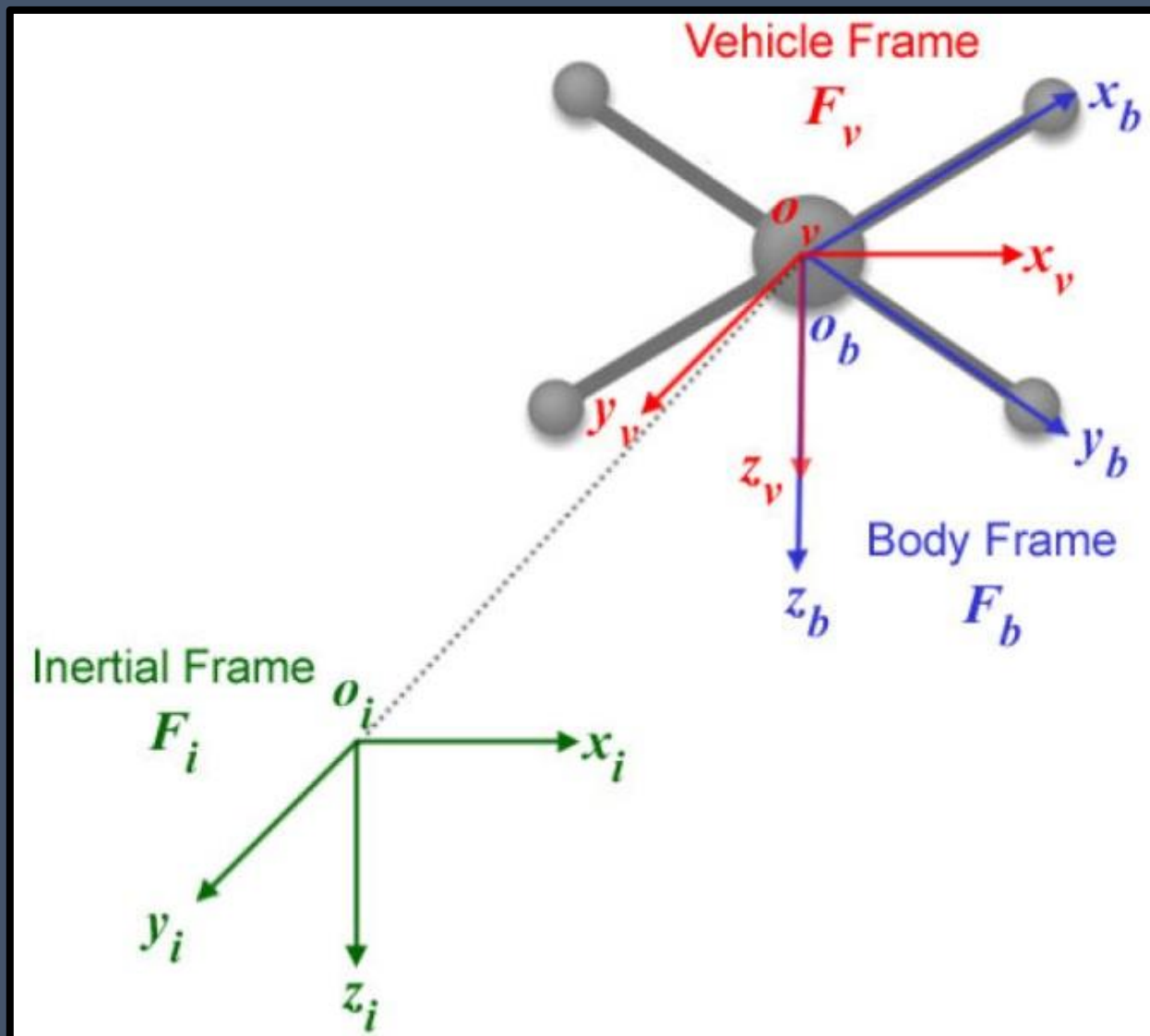
來談談數學和物理

設計構型(Configuration)

- 決定飛行器的姿態
 - 座標系定義
 - sensor frame, body frame
 - 決定螺旋槳轉向
 - 維持角動慣量為零，防止自旋
- 構型
 - 不同的構型會決定控制的不同
 - X
 - +

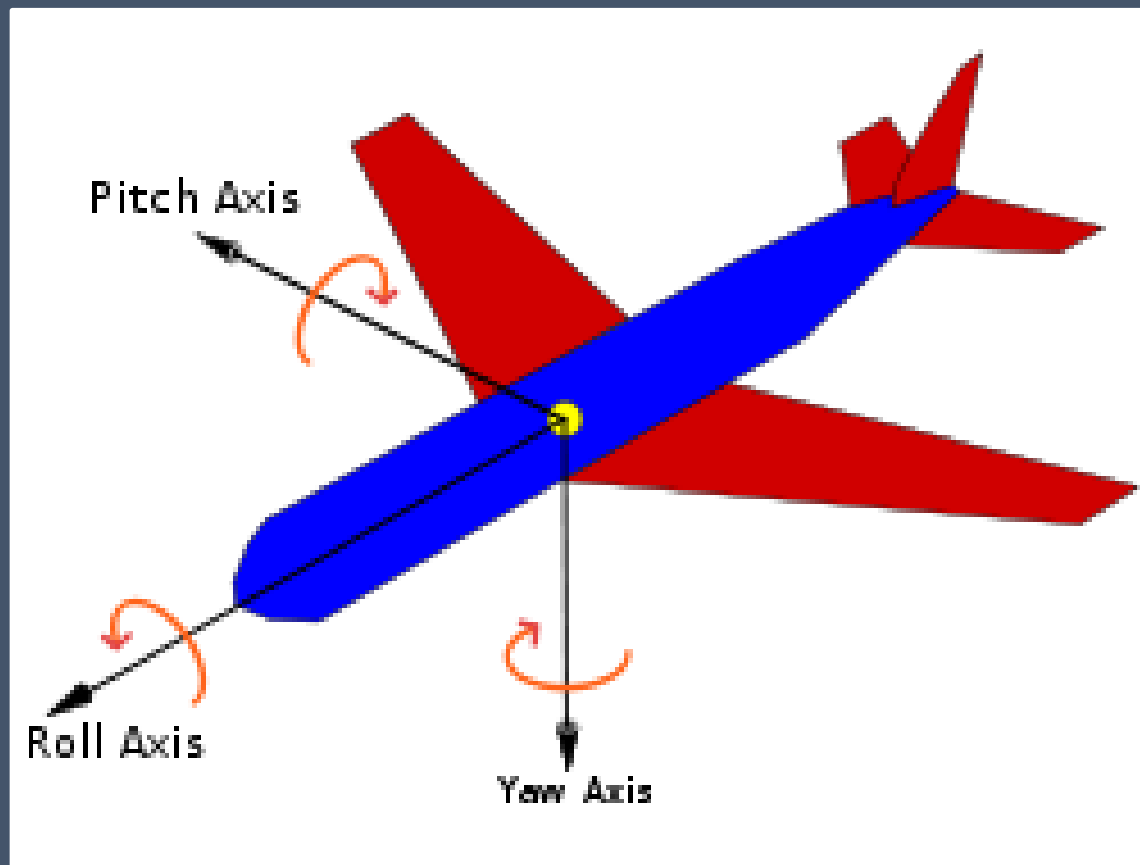
座標系定義

- 皆為Cartesian coordinate system
- Sensor frame
 - 感測器所定義的座標系
- Body frame
 - 載體座標系
 - 一般都會簡化sensor frame等同於body frame
- Inertial frame
 - 慣性坐標系
 - 相對於body frame而言不變的座標系



飛行器姿態

- 一般飛機使用的慣例



座標轉換

- 從s-frame轉換成t-frame表示為
$$\mathbf{r}^t = C_s^t \mathbf{r}^s$$

- 轉換的向量可以是
 - 位置
 - 速度(位置的導數)
 - 加速度(速度的導數)

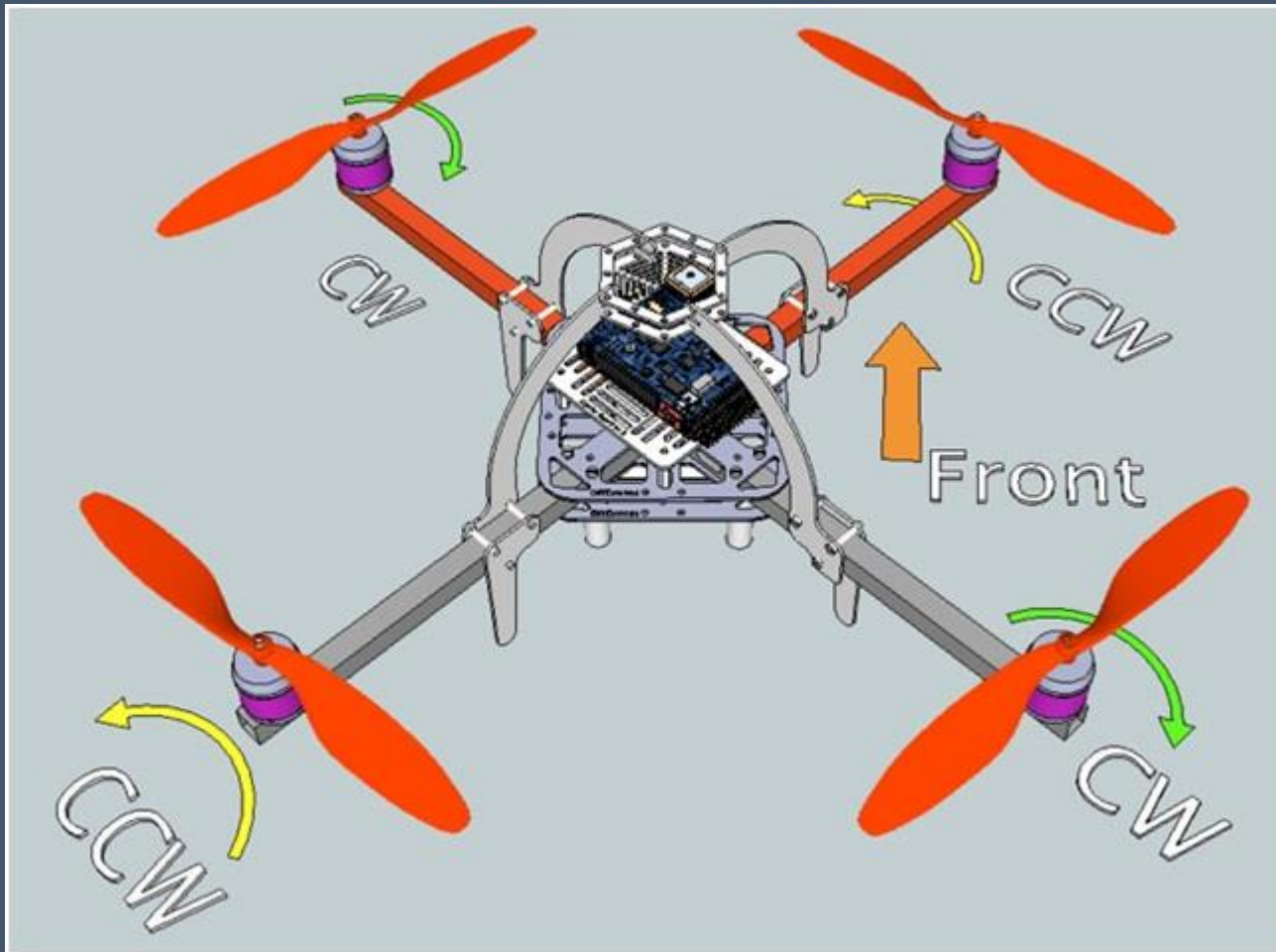
座標轉換

$$\mathbf{r}^i = C_b^i \mathbf{r}^b$$

$$C_b^i = \begin{bmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \\ c_{31} & c_{32} & c_{33} \end{bmatrix}$$

- $\mathbf{r}^i$: 3x1 vector is be expressed in inertial frame
- $\mathbf{r}^b$: 3x1 vector is be expressed in body frame
- C_b^i : 3x3 matrix is be represented to transform b-frame to i-frame

構型-X



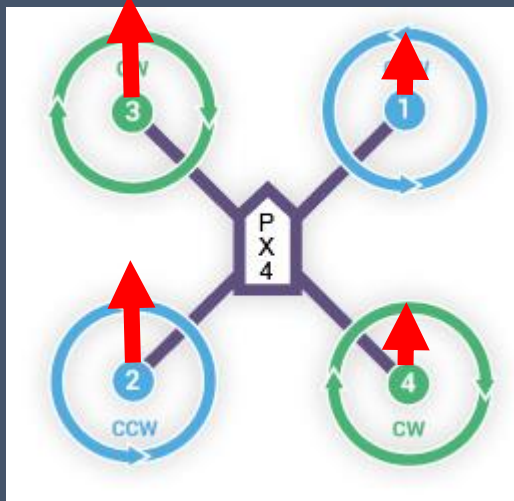
運動模型

- 沒有推導四軸的真實運動模型
 - 僅用角速度和加速度去表示

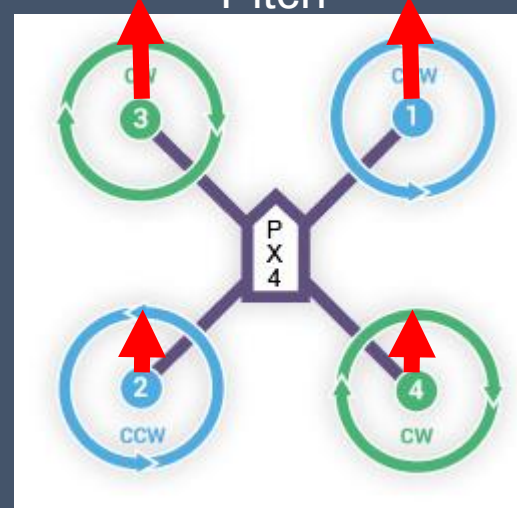
$$x = [a_x, a_y, a_z, \omega_x, \omega_y, \omega_z]^T$$

- 如何讓四軸向前？
 - 改變姿態
 - Roll angle ->左右
 - Pitch angle ->前後
 - 油門大小(throttle) ->上下
 - Yaw angle ->改變航向

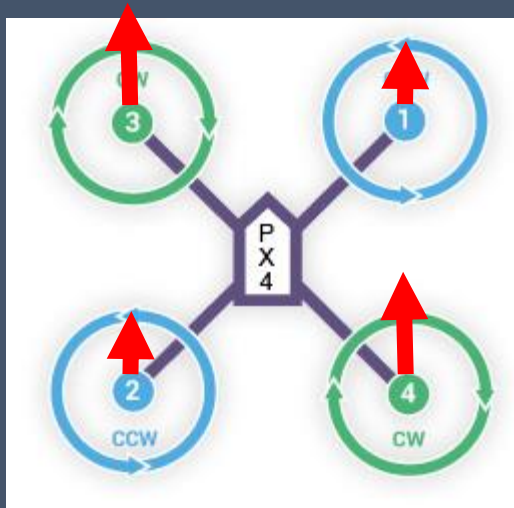
Roll



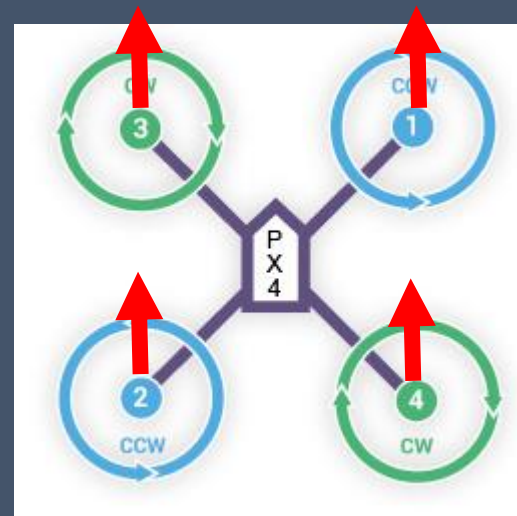
Pitch



Yaw



Altitude



數值積分

- 積分

- $\theta = \int_0^t \omega dt$: 連續時間

- 數值積分

1. Euler integration

$$x_{n+1} = x_n + \dot{x} \cdot \Delta t$$

2. Runge–Kutta(RK4) integration

$$\begin{aligned} \dot{y} = f(t, y), \quad y(t_0) = y_0. \quad & k_1 = f(t_n, y_n), \\ & k_2 = f(t_n + \frac{1}{2}h, y_n + \frac{h}{2}k_1), \\ & k_3 = f(t_n + \frac{1}{2}h, y_n + \frac{h}{2}k_2), \\ & k_4 = f(t_n + h, y_n + hk_3). \end{aligned}$$

如何得到姿態

- 十個自由度的感測器
 - 三軸**加速規**：加速度(a_x, a_y, a_z)
 - 三軸**陀螺儀**：角速度($\omega_x, \omega_y, \omega_z$)
 - 電子羅盤：磁力大小(B_x, B_y, B_z)
 - 氣壓計：大氣壓力(P_a)
- 如何表示姿態
 - 如何知道飛行器的傾角多少？
 - 參考座標？

如何得到姿態

- 尤拉角(Euler Angle)
- 方向餘弦矩陣(Direction Cosine Matrix)
- 四元素(Quaternion)
- 三個方式都可以計算座標轉換，且可以互相轉換

尤拉角

- 依序對三個座標軸旋轉，推出姿態
 - Φ : Roll
 - Θ : Pitch
 - Ψ : Yaw
- 由角速度得到其旋轉的變化率，進而得到其角度

$$\omega = \frac{d\theta}{dt}$$
$$\theta = \int \omega dt$$

尤拉角

- Propagation of Euler angles with time

$$\begin{bmatrix} \dot{\Phi} \\ \dot{\Theta} \\ \dot{\Psi} \end{bmatrix} = \begin{bmatrix} 1 & \sin\Phi \tan\Theta & \cos\Phi \tan\Theta \\ 0 & \cos\Phi & -\sin\Phi \\ 0 & \sin\Phi \sec\Theta & \cos\Phi \sec\Theta \end{bmatrix} \begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \end{bmatrix}$$

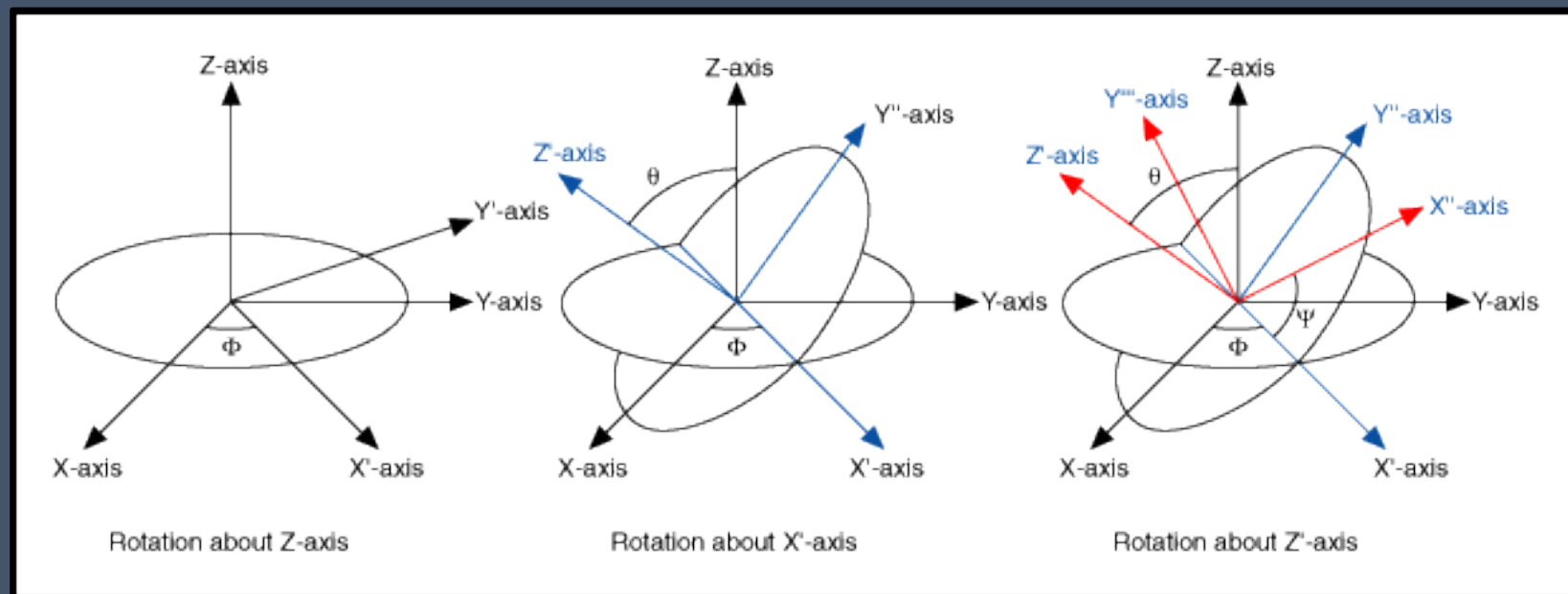
- Integration

$$\begin{bmatrix} \Phi_{n+1} \\ \Theta_{n+1} \\ \Psi_{n+1} \end{bmatrix} = \begin{bmatrix} \Phi_n \\ \Theta_n \\ \Psi_n \end{bmatrix} + \begin{bmatrix} \dot{\Phi} \\ \dot{\Theta} \\ \dot{\Psi} \end{bmatrix} \cdot \Delta t$$

- Problem

- Gimbal lock: become indeterminate when $\Theta = \pm 90^\circ$

尤拉角



尤拉角

- 可由尤拉角算出轉換矩陣 C_b^i

$$\begin{bmatrix} \cos \theta \cos \psi & -\cos \phi \sin \psi & \sin \phi \sin \psi \\ \cos \theta \sin \psi & \cos \phi \cos \psi & -\sin \phi \cos \psi \\ -\sin \theta & \sin \phi \cos \theta & \cos \phi \cos \theta \end{bmatrix}$$

方向餘弦矩陣

- 由先前座標轉換所得到的轉換矩陣 C_b^i

$$C_b^i = \begin{bmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \\ c_{31} & c_{32} & c_{33} \end{bmatrix}$$

$$\Phi = \tan^{-1}\left(\frac{c_{32}}{c_{33}}\right)$$

$$\Theta = \sin^{-1}(-c_{31})$$

$$\Psi = \tan^{-1}\left(\frac{c_{21}}{c_{11}}\right)$$

四元素

- 以一個4x1的向量表示旋轉矩陣

$$\mathbf{q} = \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} = a + b\mathbf{i} + c\mathbf{j} + d\mathbf{k}$$

$a, b, c, d \in \mathbf{R}$

- 複數(complex)的擴展數系
- 長度必須等於一(單位向量)
- 解決gimbal lock
- 節省座標轉換時間(只有乘法和加法)

四元素

- 利用參考的姿態角初始化

$$a = \cos\left(\frac{\Phi}{2}\right) \cos\left(\frac{\Theta}{2}\right) \cos\left(\frac{\Psi}{2}\right) + \sin\left(\frac{\Phi}{2}\right) \sin\left(\frac{\Theta}{2}\right) \sin\left(\frac{\Psi}{2}\right)$$

$$b = \sin\left(\frac{\Phi}{2}\right) \cos\left(\frac{\Theta}{2}\right) \cos\left(\frac{\Psi}{2}\right) - \cos\left(\frac{\Phi}{2}\right) \sin\left(\frac{\Theta}{2}\right) \sin\left(\frac{\Psi}{2}\right)$$

$$c = \cos\left(\frac{\Phi}{2}\right) \sin\left(\frac{\Theta}{2}\right) \cos\left(\frac{\Psi}{2}\right) + \sin\left(\frac{\Phi}{2}\right) \cos\left(\frac{\Theta}{2}\right) \sin\left(\frac{\Psi}{2}\right)$$

$$d = \cos\left(\frac{\Phi}{2}\right) \cos\left(\frac{\Theta}{2}\right) \sin\left(\frac{\Psi}{2}\right) + \sin\left(\frac{\Phi}{2}\right) \sin\left(\frac{\Theta}{2}\right) \cos\left(\frac{\Psi}{2}\right)$$

四元素

- 以四元素表示座標轉換矩陣 C_b^i

$$\begin{bmatrix} (a^2 + b^2 - c^2 - d^2) & 2(bc - ad) & 2(bd + ac) \\ 2(bc + ad) & (a^2 - b^2 + c^2 - d^2) & 2(cd - ab) \\ 2(bd - ac) & 2(cd + ab) & (a^2 - b^2 - c^2 + d^2) \end{bmatrix}$$

四元素

- Propagation of quaternion with time

$$\dot{\mathbf{q}} = \begin{bmatrix} \dot{a} \\ \dot{b} \\ \dot{c} \\ \dot{d} \end{bmatrix} = 0.5 \begin{bmatrix} a & -b & -c & -d \\ b & a & -d & c \\ c & d & a & -b \\ d & -c & b & a \end{bmatrix} \begin{bmatrix} 0 \\ \omega_x \\ \omega_y \\ \omega_z \end{bmatrix}$$

- Integration

$$\mathbf{q}_{n+1} = \mathbf{q}_n + \dot{\mathbf{q}} \Delta t$$

四元素

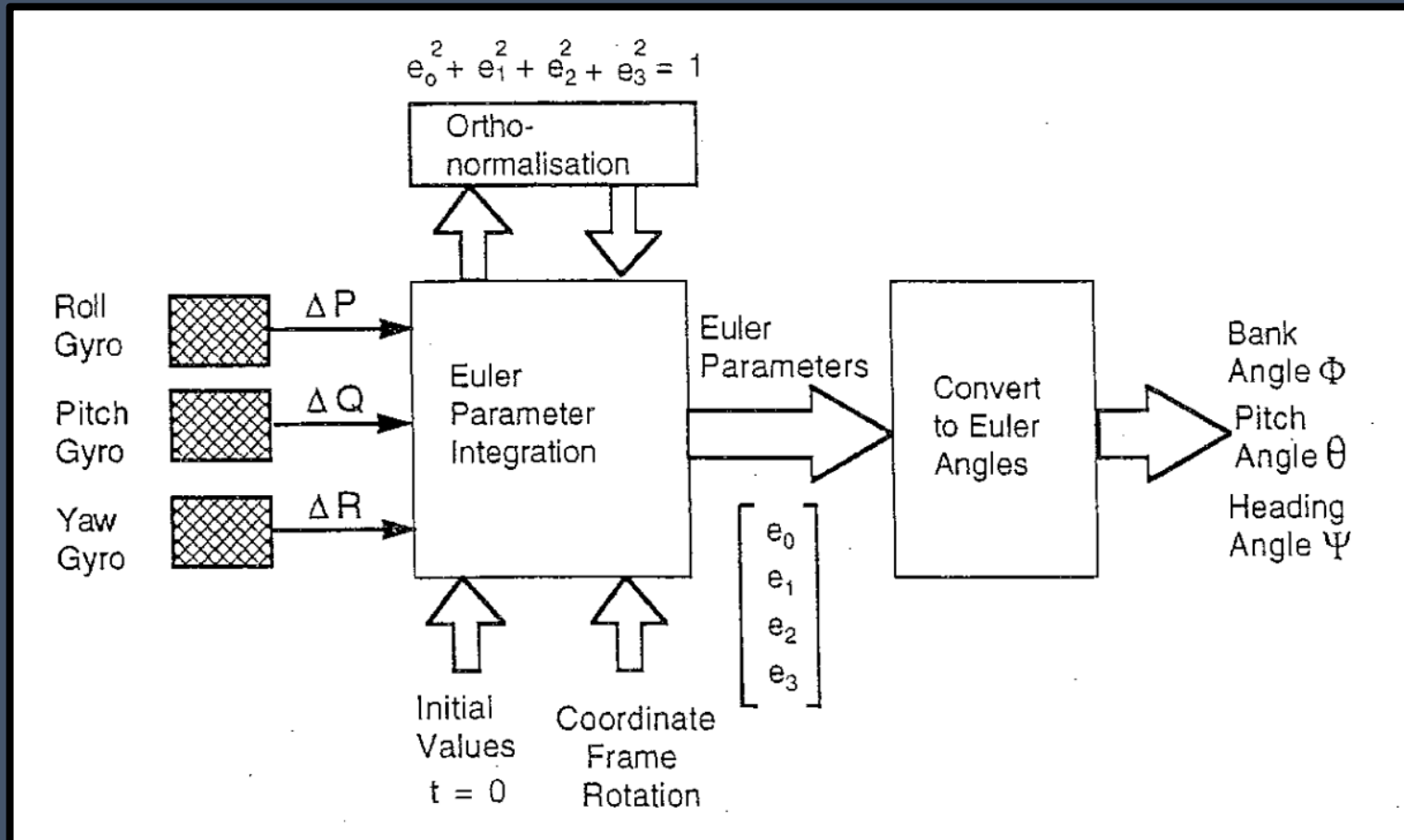
- Normalize

$$\mathbf{q}_{new} = \mathbf{q} / |\mathbf{q}|$$

- Convert back to Euler angles

$$\begin{aligned} \sin(\Theta) &= \frac{2(ac - db)}{a^2 + b^2 - c^2 - d^2} \\ \tan(\Psi) &= \frac{2(ad + bc)}{2(ab + cd)} \\ \tan(\Phi) &= \frac{2(ab + cd)}{a^2 - b^2 - c^2 + d^2} \end{aligned}$$

AHRS(Attitude and Heading reference System)



Attitude Plot



似乎這樣就解決問題了？

進入**真實**世界

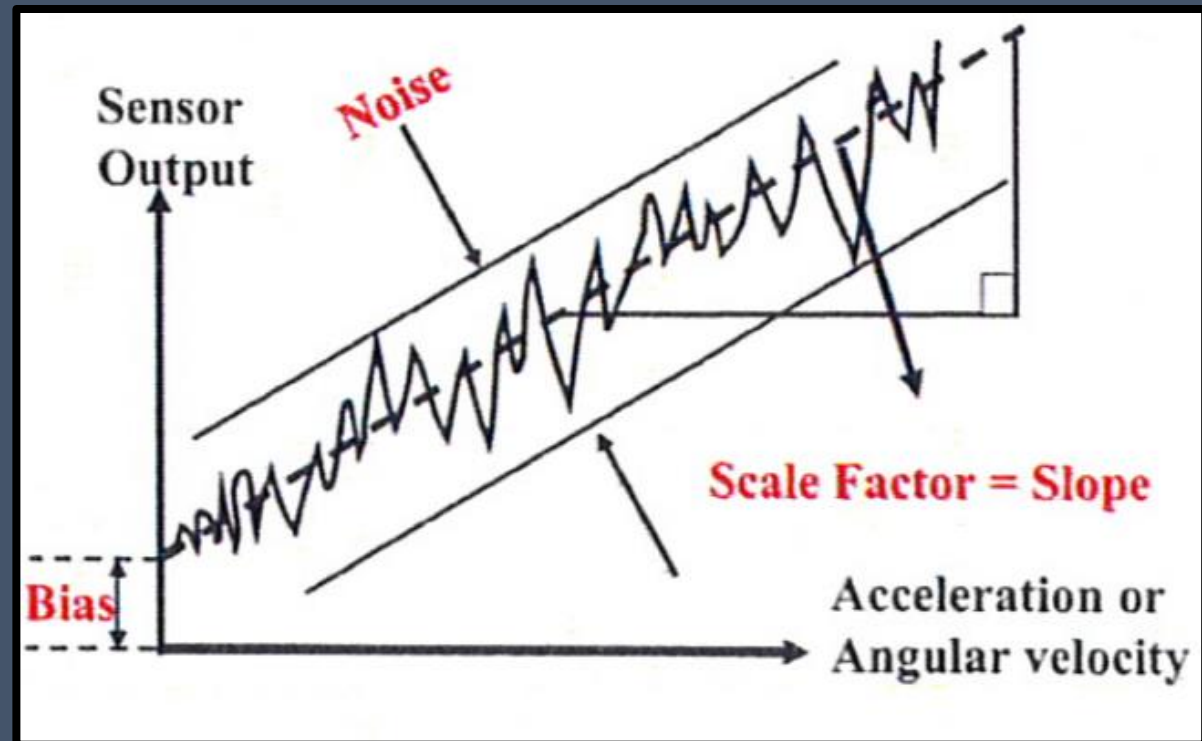
真實世界

- 感測器誤差
 - 雜訊
- 積分累計誤差
 - 因為感測器誤差所造成的放大現象
- 數值誤差
 - Euler integration or RK4
- 硬體計算能力問題
 - 硬體浮點數
 - 數學函數($\cos(x)$, $\sin(x)$...)

感測器誤差

- Sensor model

- $x = (1 - S_x)x_{true} + bias + n$
- x : measurement
- S_x : scale factor
- x_{true} : true value
- $bias$: sensor bias
- n : white noise



累積誤差

- 實際例子: MPU-9150 gyroscope
- Gyroscope noise root mean square

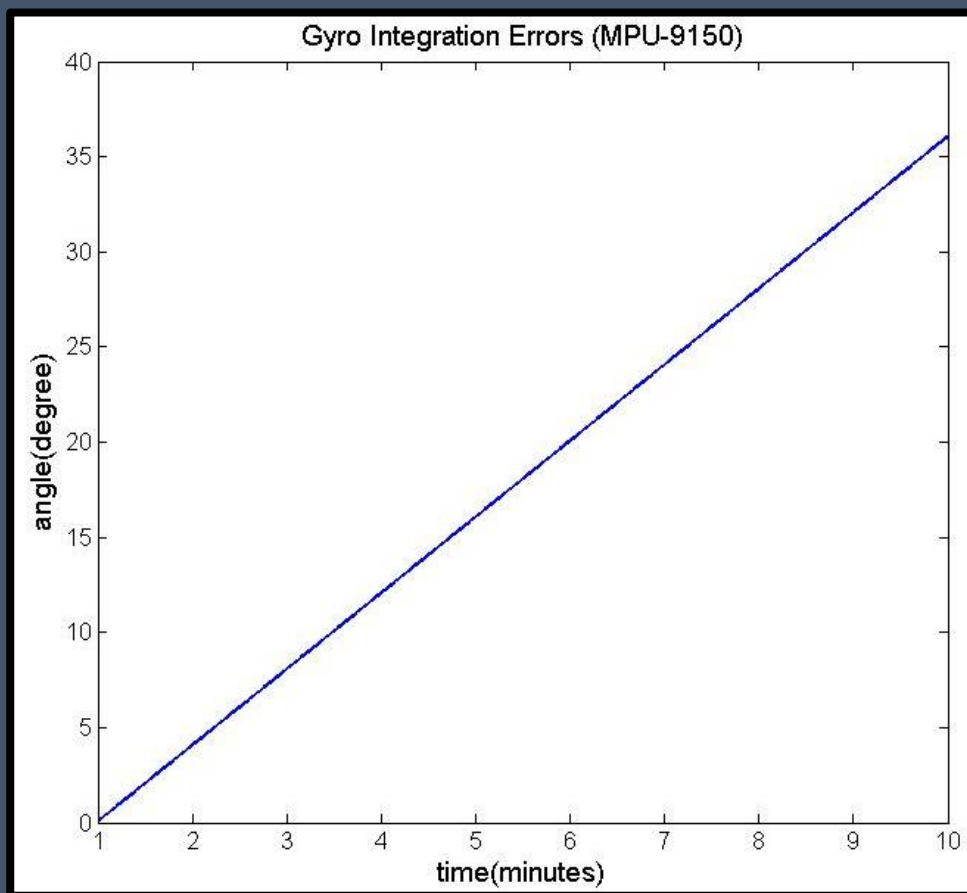
GYROSCOPE NOISE PERFORMANCE		FS_SEL=0				
Total RMS Noise		DLPFCFG=2 (92Hz)		0.06	%s-rms	
Rate Noise Spectral Density		At 10Hz		0.005	%s/ $\sqrt{\text{Hz}}$	

- 誤差計算

$$error(k + 1) = (error(k) + rms) \times \Delta t$$

累積誤差

- 因為感測器誤差導致的結果
- 以此例來說，最後算出來的角度已經達到35度以上的誤差
- 二次曲線



去除誤差

- $x = (1 - S_x)x_{true} + bias + n$
 - S_x : datasheet一般都會給
 - $bias$: 可以經由校正去除
 - n : 大自然熱雜訊，無法完全去除
- 如何去除白雜訊(white noise)?
 - 設計符合均方根誤差(root mean square error)最小的濾波器

初始校正

- 去除bias

開機時經過一段時間後，利用平均濾波器(simple moving average)得到bias

- Simple moving average
 - 類似一個低通濾波器(low-pass filter)
 - 選擇一個window size n

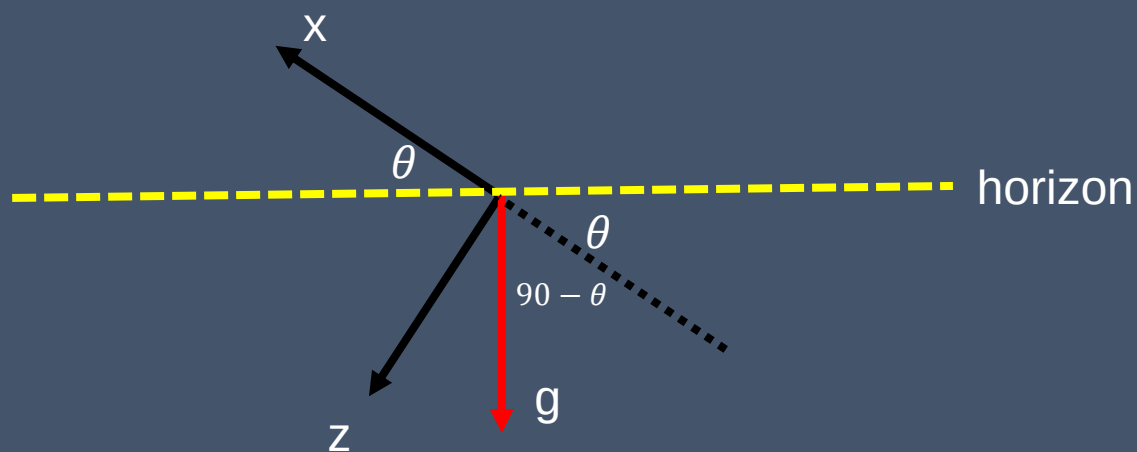
$$SMA = \frac{p_M + p_{M-1} + \cdots + p_{M-(n-1)}}{n}$$

設計濾波器

- 廣義地說，只要是去除我們不想要的資訊都可以稱之為濾波器(filter)
- 濾波器選擇
 - Moving average
 - Complementary filter
 - Kalman filter

互補式濾波器

- 目標
 - 從加速規及陀螺儀得到Pitch和Roll
 - 利用重力當作參考: $\sin(\theta) = \frac{|\mathbf{x}|}{|\mathbf{g}|}$



- 選擇權重與陀螺儀互補

$$y = (1 - w)a + wb$$

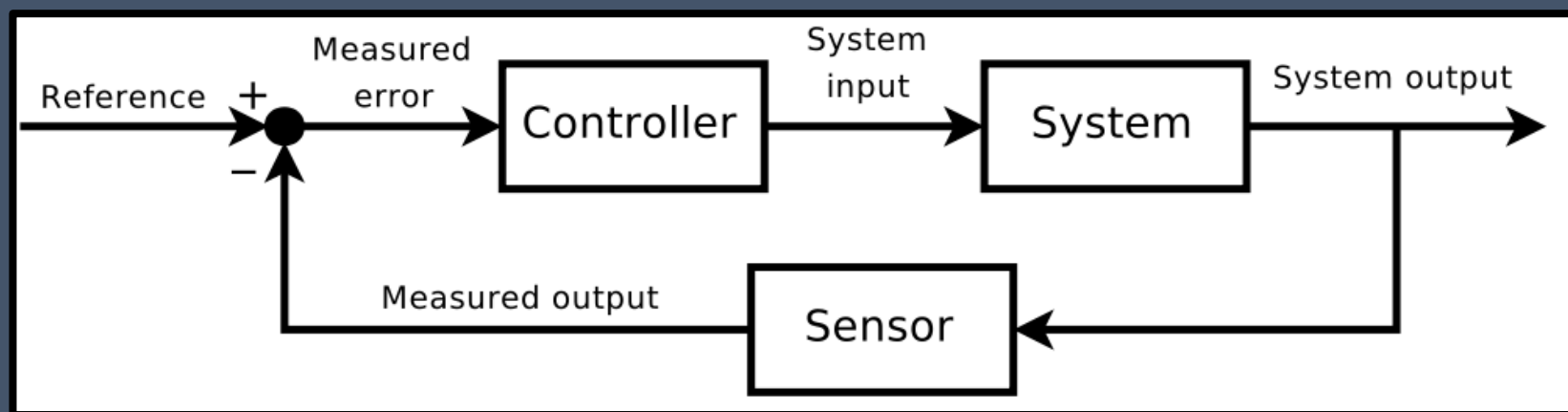
控制

控制器

- 要能起飛的第一個條件
 - Roll Angle :不會側翻
 - Pitch Angle:不會前翻
 - Yaw Angle
 - 在原本設計上yaw angle應該不會自旋，但是有可能因為推力不平均造成會慢慢的自旋

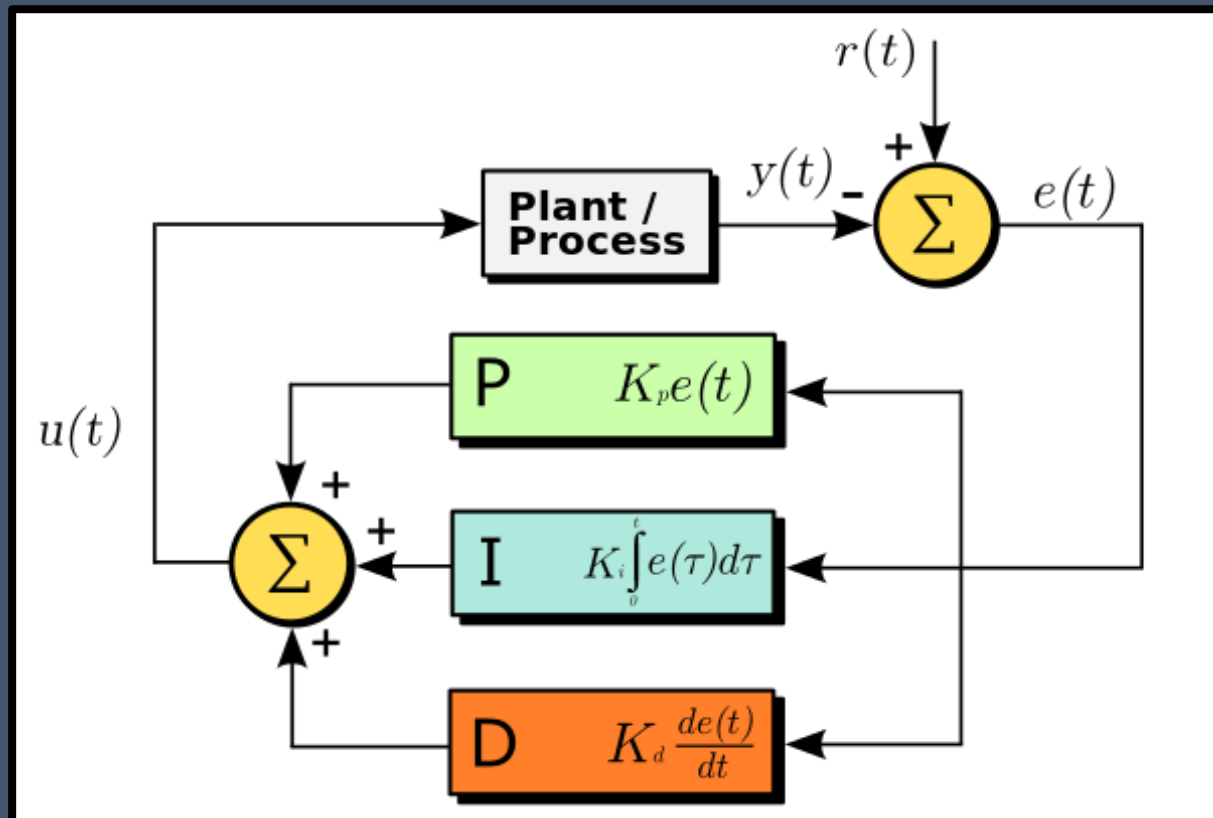
控制系統

- 典型的回授控制系統



PID控制器

- 比例(P)積分(I)微分(D)控制器
 - 一個常見的回授控制系統



數位化PID控制

- 離散化

$$u(t) = K_P e(t) + K_I \sum_{t=0}^t e(t) + K_D (e(t) - e(t-1))$$

PID控制Pseudocode

```
previous_error = 0
integral = 0
start:
    error = setpoint - measured_value
    integral = integral + error*dt
    derivative = (error - previous_error)/dt
    output = Kp*error + Ki*integral + Kd*derivative
    previous_error = error
    wait(dt)
    goto start
```

姿態PD控制

- 採用PD控制
 - 能夠夠快的反應達到set point
- $r(t)$:desired set point
 - Roll, Pitch, Yaw的預期控制角度
- $y(t)$:measure value
 - 從AHRS得到的測量角度
- $e(t) = r(t) - y(t)$:error
 - 預期的控制減去量測角度
- $e(t) - e(t - 1) = \frac{de}{dt} = \text{angular rate}$
 - 可以直接從陀螺儀的轉速得到此項

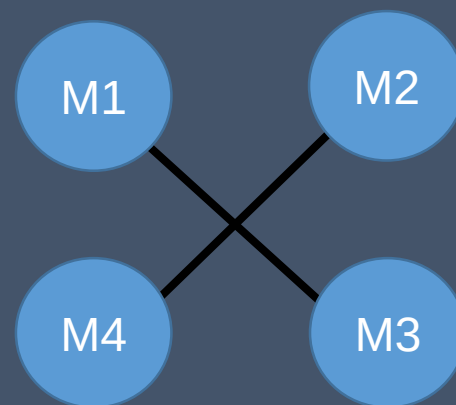
姿態PD控制

- 如何調整控制器至穩定
 - 把控制系統當作一個未知的系統(black box)
 - Manual tuning
 - 先固定 K_p 之後調整 K_d 看收斂速度

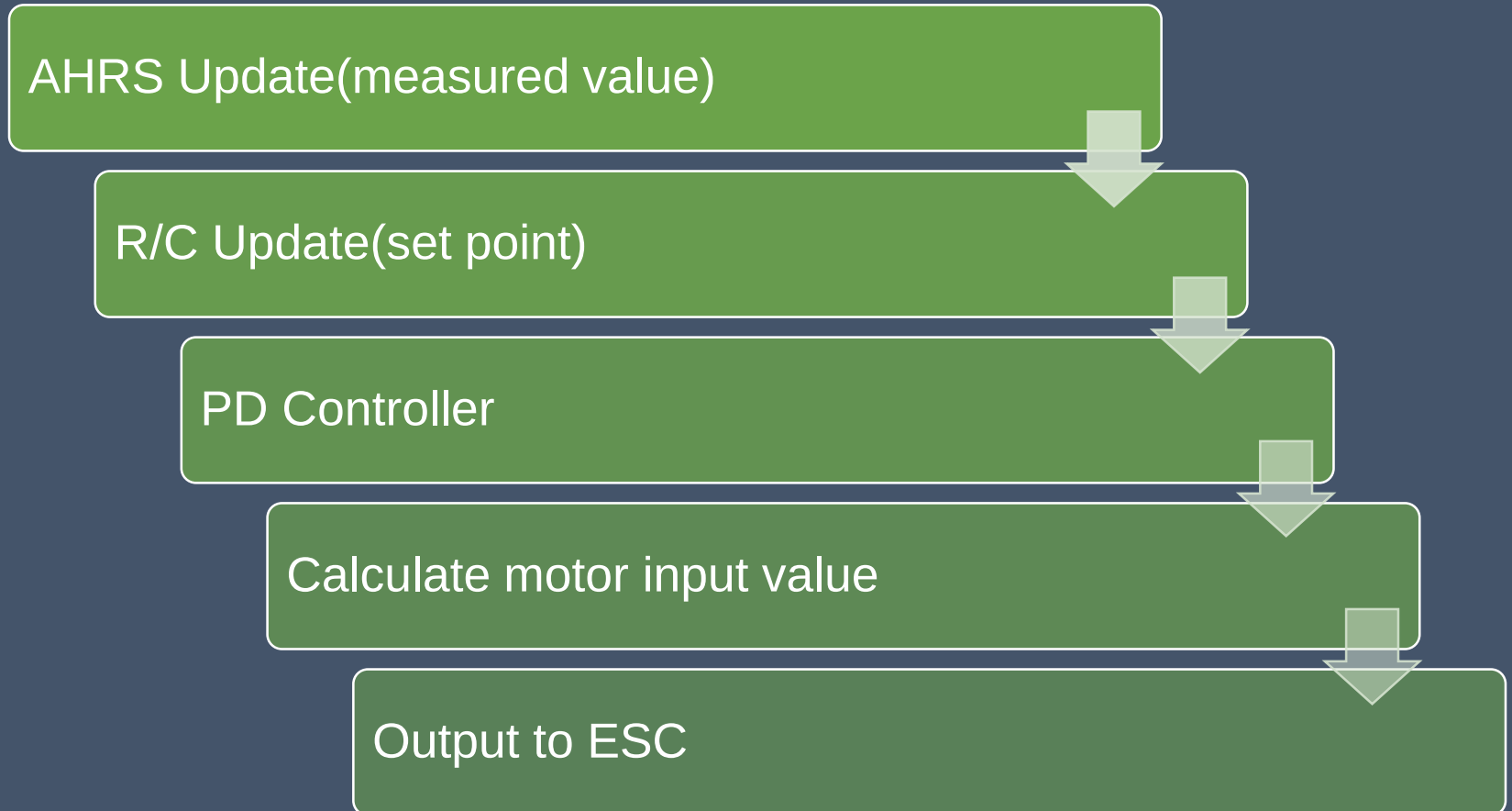
Effects of increasing a parameter independently					
Parameter	Rise time	Overshoot	Settling time	Steady-state error	Stability
K_p	Decrease	Increase	Small change	Decrease	Degrade
K_i	Decrease	Increase	Increase	Eliminate	Degrade
K_d	Minor change	Decrease	Decrease	No effect in theory	Improve if K_p small

輸出至電子變速器

- 透過電子變速器可以輸出三相交流電給一般的無刷馬達
- $M1 = \text{Throttle} - \text{Roll} + \text{Pitch} - \text{Yaw}$
- $M2 = \text{Throttle} + \text{Roll} + \text{Pitch} + \text{Yaw}$
- $M3 = \text{Throttle} + \text{Roll} - \text{Pitch} - \text{Yaw}$
- $M4 = \text{Throttle} - \text{Roll} - \text{Pitch} + \text{Yaw}$



飛控流程



安全機制

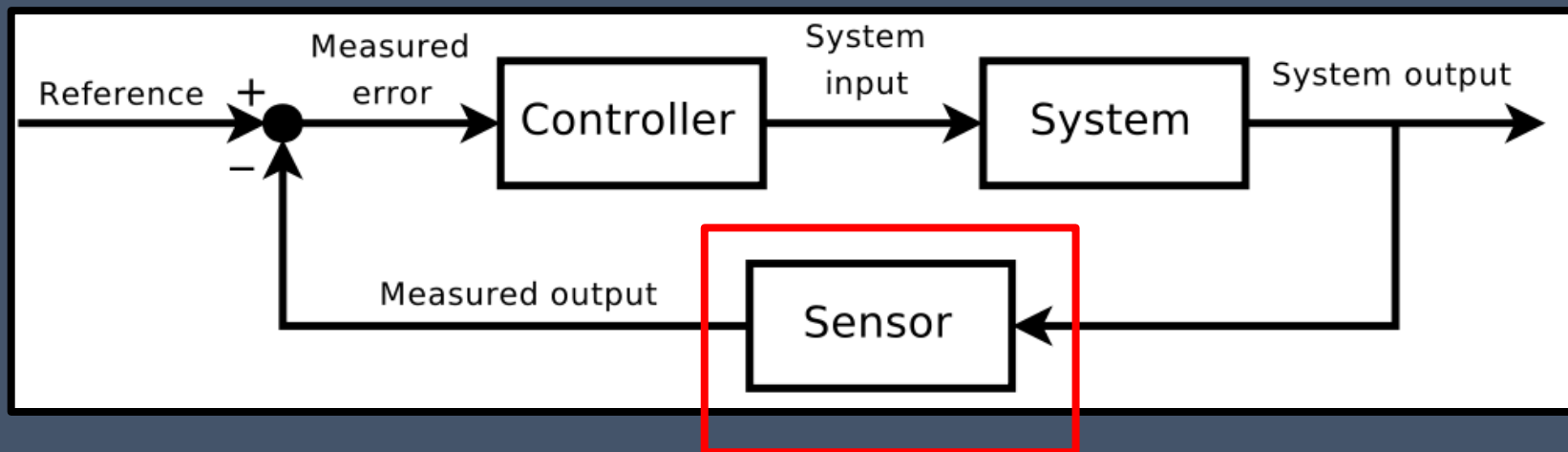
- Bound
 - 在每一個階段都有個巨集(Macro)去保護輸出的值
 - 限制在某一區間(例如pitch angle從-60~+60度)
- 遙控器油門歸零開關
 - 停止馬達運轉

姿態控制

- 目前只能保證飛行器的姿態在set point
 - 無法保證飛行器不會飄移
- 飄移問題
 - 會上下以及左右飄移
 - 沒有速度以及位置控制
 - 必須要有人為操作抵銷飄移現象

速度以及位置控制系統

- 問題在於如何取得精準的速度以及位置的資訊



機載感測器

- 必須要夠小及夠輕
- 加速規+陀螺儀
 - 高精度取得非常昂貴
- 氣壓計
 - 高度
- 超音波
 - 精度
 - 易被干擾

電腦視覺

- 通常都需要大量計算
- Visual SLAM(Simultaneous localization and mapping)
 - PTAM(*Parallel Tracking and Mapping for Small AR Workspaces*)
 - 位置以及速度
- Optical flow
 - 速度
- Kinect
 - 除了顏色(RGB)還可以得到景深(D)
 - 室外無法使用?

無線電感測器

- 室外
 - GPS
 - 可以得到高精度的速度以及可接受的位置
 - 3G, 4G
- 室內
 - WIFI
 - Bluetooth

飛行任務(task)

- 總結一下MCU要做的事情
 1. 慣性感測元件初始校正
 2. 傳送一些可用的訊息給地面接收站
 3. 從遙控器的接收器解出我們預期的姿態
 4. 更新目前姿態以及根據遙控器預期的姿態控制
 5. SD卡儲存飛行紀錄

原本分配工作的方法

- Switch case 配合systick中斷
 - 在原本的main有一個會切換switch case的無窮迴圈
 - 定時觸發systick進入中斷執行飛控任務
- 缺點
 - 缺乏彈性
 - 必須等待切換



作業系統

引入作業系統-FreeRTOS

- FreeRTOS
 - 支援多種MCU
 - 豐富社群開發經驗
- 中斷
 - 必須符合FreeRTOS的使用方式
 - 要確保某些特定的中斷不能被作業系統延遲

處理FreeRTOS中斷

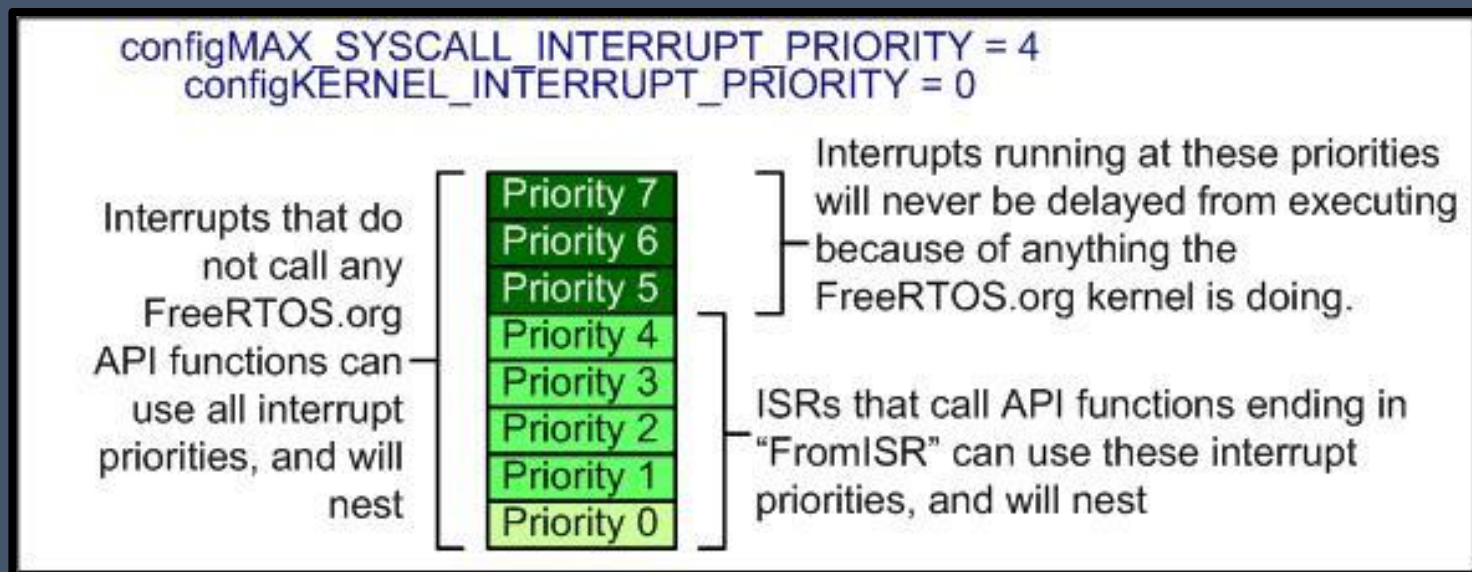
- FreeRTOS的Task優先權(數字越大擁有越高優先權)和Cortex-M的中斷優先權(數字越小擁有越大優先權)是剛好顛倒
- Cortex-M架構的MCU使用只用搶佔優先權(preemption priority)沒有次要優先權(sub priority)的中斷

處理FreeRTOS中斷

- 必須先定義優先權
- **configKERNEL_INTERRUPT_PRIORITY**
 - sets the interrupt priority used by the RTOS kernel itself
- **configMAX_SYSCALL_INTERRUPT_PRIORITY**
 - sets the highest interrupt priority from which interrupt safe FreeRTOS API functions can be called(FromISR API)

處理FreeRTOS中斷

- 本例為中斷優先權數字越大代表擁有越高優先權



處理FreeRTOS中斷

- 飛控板所需要的中斷(除了FreeRTOS本身所需)
- R/C 訊號
 - 需要五個通道的timer input capture: 占用兩個timer interrupt
 - 設定優先權比**KERNEL_INTERRUPT**也比**MAX_SYSCALL_INTERRUPT_PRIORITY** 高
- Shell
 - USART
 - 利用FromISR API
 - Queue
 - Binary semaphore

Binary Semaphore

- 實作SD卡存檔和讀檔所遇到的問題
 - 如何讓一個task維持blocked等到另一個task喚醒?
- Binary semaphore
 - task執行後就會等待semaphore(blocked)
 - 只有在得到semaphore的時候執行給予的任務(running)
 - 執行完後繼續等待下一個semaphore(blocked)
- SDIO task
 - 執行後等待來自shell的semaphore(blocked)
 - 得到semaphore後執行讀檔或存檔(running)
 - 執行完後等待來自shell的semaphore(blocked)

Resource

- 獲取感測器或使用某些傳輸介面出現的問題
 - 如何確保只有一個執行緒得取得某些資源(SPI, I2C, SDIO)?
 - 如何確保只有特定的權限才得以改變某些資源的行為?
 - 如何正確地傳遞訊息?(感測器數值,控制參數....)

Debug

- FreeRTOS assert()
 - 當移植FreeRTOS到飛控板的時候會發生程式執行到某些地方卡住
 - 如果以openocd配合gdb觀察會發現FreeRTOS會卡在某個無窮迴圈，而通常會有說明為何卡在這裡的註解
- 常見的問題
 - Heap1~4選擇
 - Task設計成有可能跳離無窮迴圈，但是並沒有呼叫FreeRTOS刪除Task的API

測試

- 裝上槳葉前
 - 確認所有功能正常運作
- 對飛控有做修改後
 - 必須測試姿態計算的極限
- 執行飛控的task
 - 利用示波器配合GPIO Toggle
- 目前找不到很方便的方法可以自動測試
 - Unit testing
 - CI(Continuous integration)

總結

- 姿態計算，濾波器設計以及控制器設計是個綁在一起的問題。如果有辦法改善其中之一，就有辦法讓飛行控制做的更完善
- 如果想不須倚靠駕駛員，就必須完成速度以及位置控制以達成自動導航
- 利用作業系統管理系統資源，是大多數飛控板都想達成的目標
- 測試問題
- 安全!

更多參考資料

- <http://blog.tkjelectronics.dk/2012/03/quadcopters-how-to-get-started/>
- <http://blog.oscarliang.net/build-a-quadcopter-beginners-tutorial-1/>
- <http://www.facstaff.bucknell.edu/mastascu/econtrolhtml/intro/intro1.html>
- <http://andrew.gibiansky.com/blog/physics/quadcopter-dynamics/>